ORIUS Professor Appendix B: Empirical, Runtime, and Artifact Surface

Author details omitted for professor review draft

I. Appendix Overview

This appendix carries the empirical and systems depth behind the main paper: battery witness detail, universal benchmark semantics, runtime budgets, CertOS lifecycle tables, protocol cards, and one repository maturity visual. It is deliberately artifact-grounded and reuses the same publication surfaces that support the canonical dissertation, rather than local dashboard snapshots or independently written summaries.

II. Runtime, Latency, and Governance Evidence

This appendix section collects the evidence surfaces that justify the runtime contract in the main IEEE draft. The purpose is not to flood the reader with every locked artifact in the repo. It is to show that the adapter, benchmark, and CertOS claims are backed by measured runtime latency, explicit replay tables, and a visible governance gate.

A. Latency and Footprint

The latency story is straightforward. The runtime chapter (`chapters/ch22_latency_systems_footprint.tex`) measures the DC3S pipeline with `time.perf_counter_ns()` and reports sub-millisecond step time on the locked battery dispatch surface. The key component latencies are the reliability scorer, the drift updater, the uncertainty set builder, and the action repair stage.

The same section also provides the best compact footprint statement for a professor-facing draft: the full step fits comfortably inside a 30-second dispatch cycle, and the runtime memory proxy remains dominated by the forecast and residual buffers rather than by certificate bookkeeping. That makes the runtime layer operationally cheap enough to be believable as a Physical AI safety wrapper.

B. Runtime Budget Across Domains

The cross-domain runtime-budget matrix in `reports/publication/tbl_orius_runtime_budget_matrix.tex` is the best evidence that the runtime contract is not battery-specific. It shows that the six-domain program preserves sub-millisecond P95 latency while keeping certificate rates at 100 percent on all rows.

This table should stay close to the runtime section in the professor draft because it directly supports the claim that the universal runtime layer is lightweight across domains while still respecting the evidence tier.

C. Governance Evidence

The governance story is best captured by the locked lifecycle matrix in `reports/publication/tbl_orius_governance_lifecycle` and the domain-closure matrix in `reports/publication/orius_domain_closure_matrix.csv`. Together they show that CertOS is not a decorative abstraction: the same issue/validate/expire/fallback logic is observed in the defended rows, while navigation and aerospace remain gated by explicit blockers.

The appendix should end with the governance boundary spelled out plainly: the runtime layer is fully visible, the certificates are auditable, and the fallback path is explicit, but the claim remains tiered by domain. Battery is the reference witness; AV, industrial, and healthcare are defended bounded rows; navigation and aerospace remain gated. That is enough for a strong IEEE draft, and it is more defensible than flattening the evidence into one universal-number story.

D. Repository maturity visual

Appendix B is also the right place for one explicit repository-level visual. The professor draft should show the reader that ORIUS is not only a narrative proposal but a maintained research system with a real public code surface and a generated architecture map.

III. Appendix B: Extended Battery Evidence

This appendix expands the battery witness with the full publication-facing empirical package: controller rankings, reliability diagnostics, raw-sequence track results, operational traces, and the failure-analysis surfaces that support the compact main-text claim.

TABLE I: Per-component DC3S latency summary from the locked runtime study.

Component	Mean	Median	P95	P99	Max
Reliability scoring (OQE)	0.026	0.021	0.043	0.130	0.712
Drift update (Page-Hinkley)	0.000	0.000	0.001	0.001	0.160
Uncertainty set build (RUI)	0.012	0.011	0.011	0.047	0.724
Action repair (SAF/Shield)	0.002	0.002	0.002	0.003	0.210
Full DC3S step	**0.042**	**0.037**	**0.041**	**0.193**	**2.416**

TABLE II: Runtime and governance summary for the professor-facing ORIUS draft.

Domain	Row	P95 latency (ms)	Repair (%)	Fallback	Audit (%)	Current limit
Battery Energy Storage	Witness row	0.1	n/a_reference	safe_hold	97.0	external bench and field telemetry packaging still remain bounded deployment items
Autonomous Vehicles	Defended bounded row	0.2	100.0	full_brake	94.0	promoted
Industrial Process Control	Defended bounded row	0.2	91.7	power_cap	94.0	industrial_train_validatio
Medical and Healthcare Monitoring	Defended bounded row	0.1	16.7	max_alert	94.0	healthcare_train_validatic
Navigation and Guidance	Shadow-synthetic	0.1	100.0	hold_position	58.0	navigation_kitti_runtime_
Aerospace Control	Experimental	0.2	18.8	envelope_hold	55.0	aerospace_realflight_runti

A. Locked evaluation scope

The battery evidence used in the professor-facing manuscript comes from the locked CPSBench package with 168-hour episodes, seed-controlled runs, and fault families covering dropout, drift, delay, stale sensing, and combined degraded telemetry. The main result is not that every baseline is competitive; it is that the ORIUS repair-aware variants eliminate true-state violations under the same locked protocol that exposes failures in the non-repair baselines.

B. Controller ranking on the publication leaderboard

Table IV records the battery leaderboard as published in the repo. The zero-violation tier is occupied by the scenario methods and the ORIUS repair-aware variants, while the deterministic and static interval baselines are separated by their non-zero violation rates.

This ranking is useful because it separates two questions that are often conflated. Rank is driven by both safety and cost, while the thesis claim is only about safety under degraded observation. On that narrower question, the important fact is that the ORIUS variants stay in the zero-violation tier and the deterministic/static baselines do not.

C. Reliability-conditioned baseline audit

Table V expands the same result family with coverage and interval-width diagnostics. The ORIUS FTIT variant is the most conservative but also the most coverage-stable. The wrapped variant is less conservative, but it still preserves zero true-state violations. That tradeoff is exactly what the professor-facing manuscript emphasizes: safety is robust, while efficiency is controller-dependent.

The appendix table also makes clear why the paper should not oversell the deep OQE itself. The deep reliability model is a useful input to the repair pipeline, but the published summary shows only modest fault separability (`fault_auc` = 0.534) and no evidence of a dramatic low-reliability lift. In other words, the system is not relying on a miraculous classifier; it is relying on a conservative safety layer that behaves well even when the quality score is only moderately informative.

D. Deep OQE diagnostic summary

Table VI records the strongest high-level deep OQE diagnostics from the publication package.
The value of this table is not that the deep model dominates the heuristic. It does not. The value is that ORIUS remains safe without needing a perfect fault classifier: the quality signal only has to be informative enough to steer the repair-and-certify layer away from hidden violations.

E. Raw-sequence track benchmark

The raw-sequence benchmark is an important negative result because it shows how hard the unengineered sensing problem remains. Table VII compares a reliability-conditioned PatchTST model on raw sequence inputs against a GBM on engineered tabular features. Across load, wind, and solar targets, the raw-sequence model is substantially worse in RMSE and MAE, while calibration is mixed rather than uniformly superior.

TABLE III: CertOS runtime summary from the battery lifecycle demo.

Metric	Value
Total steps	96
Valid steps	47
Degraded steps	16
Fallback steps	33
Runtime interventions	33
Audit entries	96
Audit completeness	100.0%
Step latency	0.59 ms

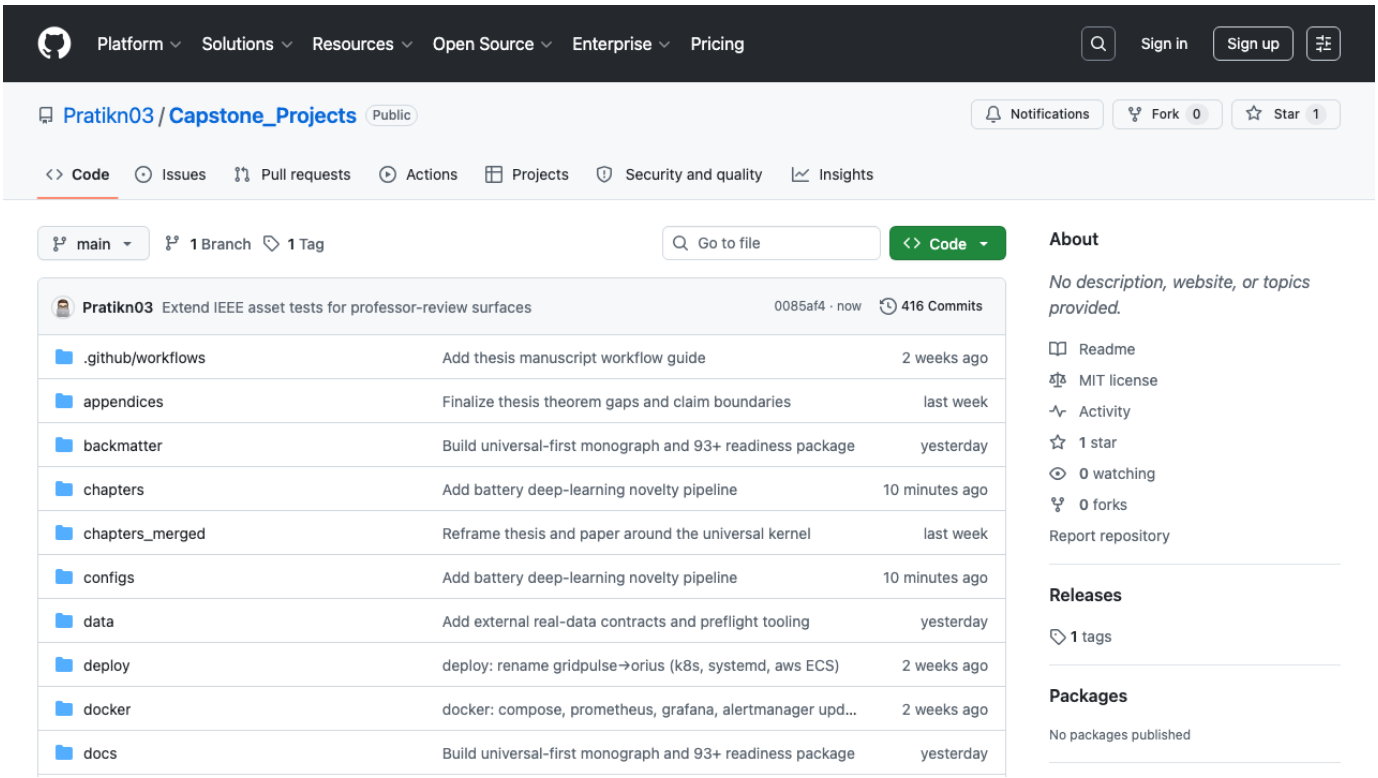

Fig. 1: GitHub-style repository overview used as a maturity visual in the professor draft. This figure is supplemental and does not carry scientific claims by itself.

F. Operational trace and failure analysis

The 48-hour operational trace and the ablation suite give the strongest mechanistic evidence. In the trace, interventions cluster around telemetry degradation rather than random dispatch variation, and the shield is otherwise transparent. In the ablation study, disabling OQE, inflation, or projection reduces the quality of the protection layer; disabling FTIT mostly increases conservatism rather than breaking safety.

- The 48-hour trace records 47 interventions over 1 680 steps, or 2.8% of steps, with clusters tied to dropout bursts and drift peaks.
- The most severe avoided violation in the trace occurs when the LP proposes a large discharge against an over-optimistic observed SOC, while the shield projects the action back to a safe discharge.
- The ablation tables show that OQE and uncertainty inflation are the safety-critical elements; FTIT is a performance refinement, not the main safety barrier.

This is the right level of detail for the professor-facing manuscript: the safety story is not a single lucky run, but a reproducible pattern across controller rankings, reliability audits, raw-sequence stress tests, and trace-level failure avoidance.

IV. Benchmark and Reproducibility Protocol

A. Why ORIUS-Bench is central to the thesis

Once the runtime object has been fixed, the next question is whether it can be measured in a way that matches the theory rather than only the software stack. ORIUS-Bench is the answer to that question. It is not infrastructure

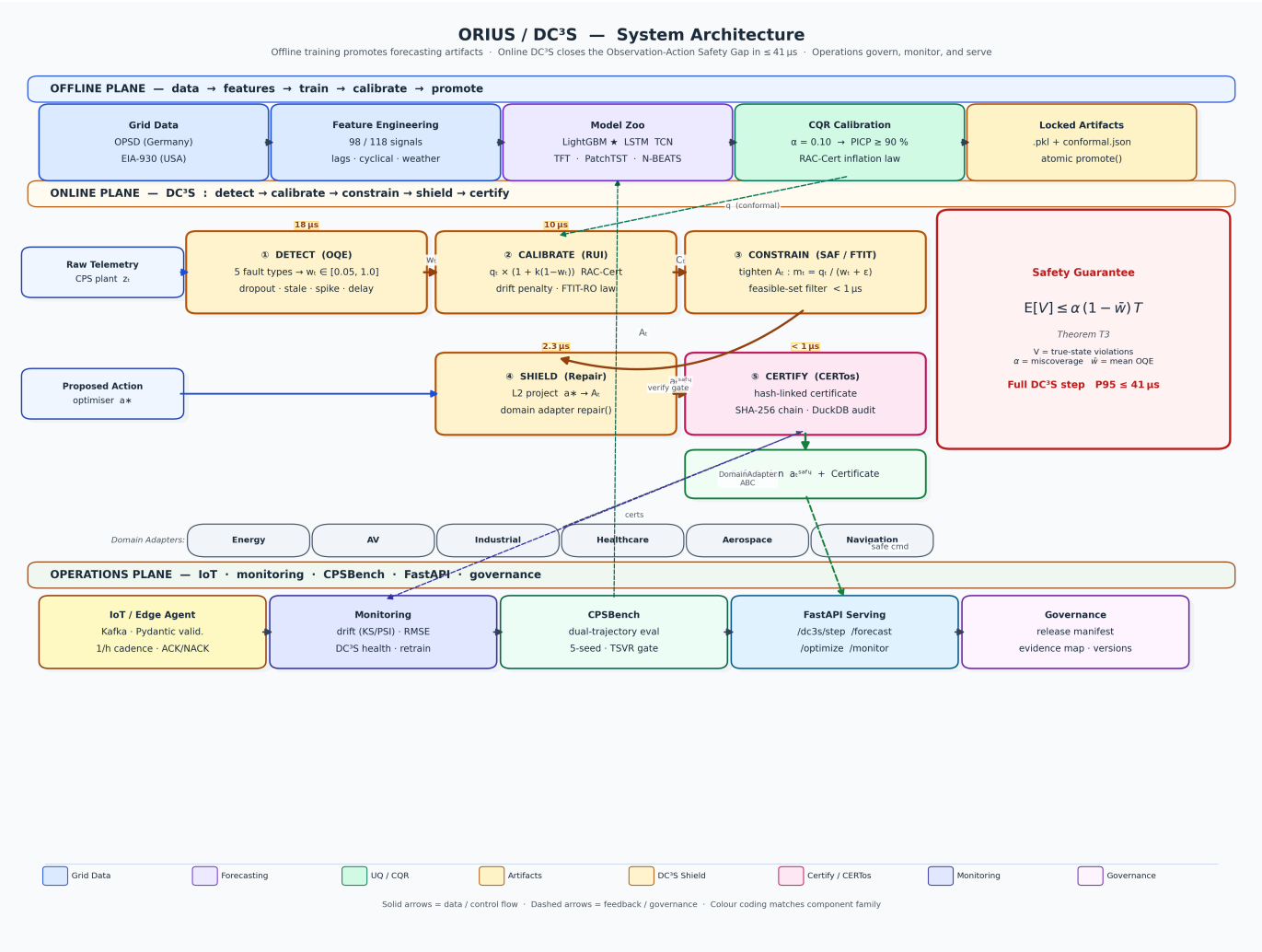

Fig. 2: Repository-backed architecture visual reused in Appendix B to connect the code surface to the runtime stack argued in the main paper.

TABLE IV: Battery leaderboard from the tracked publication artifact.

Rank	Controller	Viol. rate	IR (%)	Mean PICP_{90}
1	scenario_mpc	0.000000	0.000000	0.934722
2	scenario_robust	0.000000	0.000000	0.975000
3	aci_conformal	0.000000	2.083300	0.965278
4	dc3s_wrapped	0.000000	0.000000	0.880556
5	dc3s_ftit	0.000000	8.819400	0.930555
6	deterministic_lp	0.129861	0.000000	0.879861
7	cvar_interval	0.191667	0.000000	0.879861
8	robust_fixed_interval	0.191667	0.000000	0.879861

attached after the science; it is the measurement contract that makes the dissertation's central object observable. If OASG is the gap between observed-state legality and true-state safety, then the benchmark must expose both surfaces explicitly. A benchmark that measures only forecast error, controller cost, or observed-state success cannot validate the ORIUS claim because it cannot see the hidden failure event the book is about.

B. Dual-trajectory evaluation

Every ORIUS-Bench episode therefore carries two trajectories:

- the *observed* trajectory available to the runtime after degradation; and
- the *true* trajectory used only by the replay or evaluation harness to judge whether release was physically safe.

TABLE V: Reliability-conditioned battery baselines from the locked audit.

Method	TSVR	Obs. viol.	Coverage	Width	Repair rate
ORIUS FTIT	0.000000	0.000000	0.971627	3906.547749	0.027976
ORIUS Wrapped	0.000000	0.000000	0.884127	559.920971	0.000000
Adaptive Conformal + Repair	0.000000	0.000000	0.962500	2824.252435	0.013889
Deterministic LP	0.039087	0.039087	0.883333	503.130805	0.000000
Robust Fixed Interval	0.255754	0.255754	0.883333	503.130805	0.000000

TABLE VI: Deep OQE diagnostic summary.

Metric	Heuristic	Deep
Eval reliability MAE	0.117620	0.162228
Fault AUC	–	0.534482
Low-reliability fault lift	–	0.000000

That separation makes the benchmark stricter than a conventional controller comparison. A nominal controller may still appear coherent on the observed state while the replay harness records a true-state violation. ORIUS-Bench treats that divergence as first-class evidence.

C. Deterministic replay and fault schedules

The benchmark uses deterministic fault schedules so that comparisons are reproducible and so that certificate traces can be replayed under the same degraded-observation regime. Fault families include dropout, stale holds, delay, reordering, spikes, drift, and domain-local analogs. The scientific point is not merely that faults exist. It is that the same fault grammar can be replayed against one release contract across domains.

Strict replay discipline matters for another reason: it prevents the dissertation from hiding behind one-off operational anecdotes. A run must be reproducible through a manifest, fault schedule, and artifact family. If a result cannot be regenerated from the tracked surface, it is not part of the defended thesis claim.

D. Metrics that match the runtime object

ORIUS-Bench measures the runtime layer through seven core metrics:

TSVR true-state violation rate, the primary safety outcome;
OASG the observed-safe / true-unsafe discrepancy rate;
CVA certificate-valid action rate or validity surface;
GDQ graceful degradation quality under fallback or reduced-operation modes;
IR intervention rate, capturing how often the runtime must repair or deny release;
AC audit completeness, measuring whether release is reconstructable after the fact;
RL runtime latency, measuring whether the protection layer remains operationally usable.

This metric family matters because it keeps the book from collapsing into a single leaderboard score. ORIUS is not about RMSE alone, and it is not about controller cost alone. It is about safe release under degraded observation. The benchmark therefore measures whether release remained safe, whether intervention was required, whether the certificate remained meaningful, and whether the runtime stayed fast enough to matter.

E. Strict universal mode and legacy compatibility

The dissertation adopts a strict universal reading of the benchmark. Domain-neutral violation predicates, observed-state legality predicates, and constraint margins must flow through the adapter contract. Legacy battery-shaped compatibility fields may remain in the codebase for historical or bounded-support reasons, but they are not allowed to define the universal benchmark story in the manuscript.

That distinction keeps the witness row from quietly redefining the universal object. Battery is the deepest defended row because its evidence chain is strongest, not because the benchmark is secretly battery-specific. ORIUS-Bench must therefore be written in domain-agnostic terms even when the strongest current replay surface is battery-backed.

F. Artifact provenance and claim locking

The benchmark protocol is inseparable from the claim-lock pipeline. The canonical thesis surface is `paper/paper.tex`, but its headline tables and figures are backed by manifests, publication matrices, and governed artifact paths. This is what prevents the manuscript from becoming a persuasive story about results that cannot later be checked.

TABLE VII: Raw-sequence track benchmark from the publication artifact.

Target	Track	Model	RMSE	MAE	$PICP_{90}$
Load	Raw sequence	PatchTST-reliability	7414.479	5204.866	0.610958
Load	Engineered tabular	GBM	305.121	187.869	0.882143
Wind	Raw sequence	PatchTST-reliability	6564.913	5091.294	0.714088
Wind	Engineered tabular	GBM	219.108	129.362	0.936905
Solar	Raw sequence	PatchTST-reliability	3769.459	2533.273	0.866943
Solar	Engineered tabular	GBM	240.353	117.319	0.787698

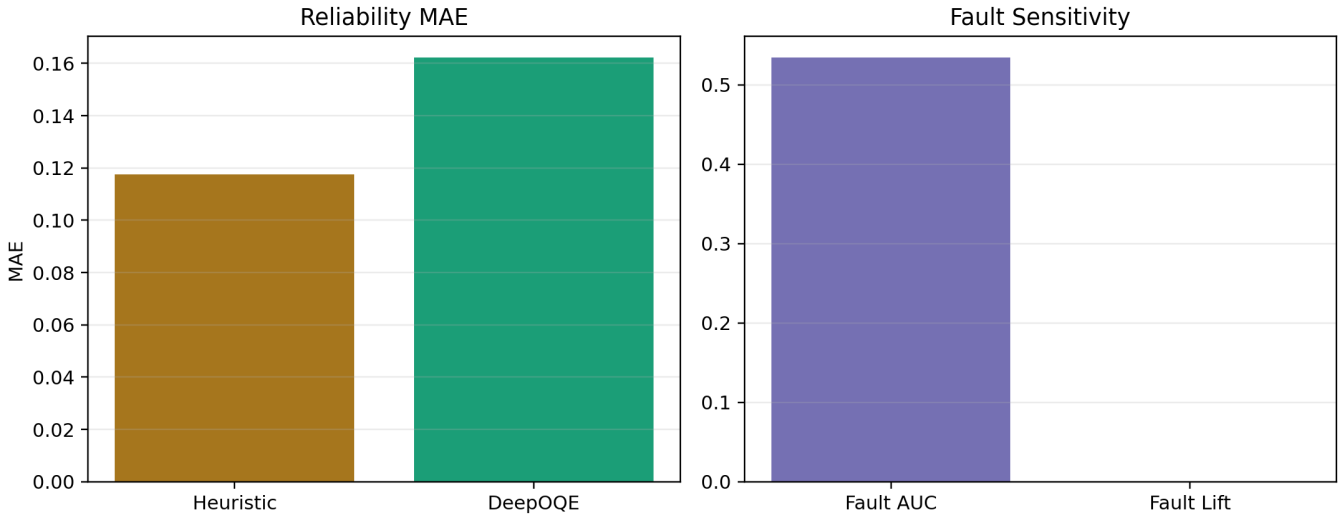

Fig. 3: Deep OQE summary plot from the locked publication package. The main takeaway is not superiority over the heuristic, but the presence of a usable quality signal that can be routed into the repair layer.

The repository's publication surfaces already enforce this discipline. The parity matrix, submission scorecard, runtime/governance matrices, and deployment-validation scope are not mere reporting layers. They are the machine-readable surfaces that keep prose aligned with evidence status.

G. Reproducibility checklist

For a result to count as defended in this dissertation, the following conditions must hold:

1) the domain row must have an explicit safety predicate and adapter semantics;
2) the replay must expose true and observed trajectories;
3) the fault schedule and manifests must be reproducible;
4) the result must land in a tracked artifact family; and
5) the claim validator must allow the corresponding manuscript language.

This checklist is stricter than "the code ran once." It is the mechanism by which ORIUS turns software artifacts into scientific evidence.

H. Benchmark takeaway

What this chapter adds to the argument is a measurement discipline equal to the runtime object introduced in Chapters 3 and 4. ORIUS-Bench makes degraded-observation safety measurable through dual-trajectory replay,

TABLE VIII: Cross-Domain OASG (Observed-Action Safety Gap) Rates.

Domain	Baseline OASG	ORIUS OASG	Reduction
Battery (Ref.)	0.1250	0.2500	-100.0%
Navigation	0.1667	0.2083	-25.0%
Industrial	0.2917	0.0000	100.0%
Healthcare	0.2500	0.0000	100.0%
Aerospace	0.2917	0.0000	100.0%
Vehicle (AV)	0.0833	0.0000	100.0%

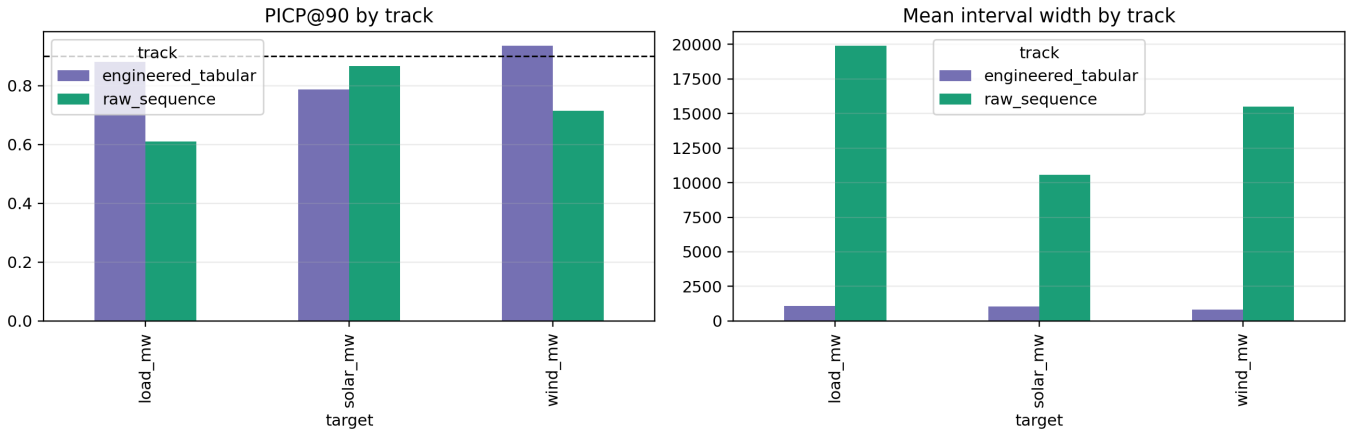

Fig. 4: Raw-sequence track benchmark from the locked publication package. The benchmark is intentionally hard: the raw-sequence model is materially worse than the engineered baseline, which is why ORIUS treats this track as a stress-test rather than a victory lap.

deterministic fault injection, runtime metrics matched to the release object, and claim-locked provenance. Equal-domain maturity still depends on which rows clear those requirements, but the measurement contract itself is now stable across the dissertation.

V. Runtime Latency and Systems Footprint

Safety guarantees are worthless if the controller cannot compute them within the dispatch cycle. This chapter benchmarks the wall-clock latency of every DC3S sub-component, characterises the runtime memory footprint, and establishes that the full pipeline fits comfortably within a 30-second dispatch interval on commodity hardware.

A. Why Latency Matters for Real-Time Dispatch

Grid-scale BESS dispatch operates on intervals of 5–30 minutes for energy arbitrage and 2–4 seconds for frequency regulation. The DC3S pipeline must complete within the shortest applicable interval. If the safety shield exceeds the cycle time, the controller either dispatches uncertified actions or stalls, both unacceptable outcomes.

B. Measurement Methodology

Latency is measured using Python `time.perf_counter_ns()` around each DC3S sub-component during a 168-hour CPSBench episode under `drift_combo` (seed 42). The episode comprises $N = 168$ hourly steps. All measurements are recorded in `reports/publication/dc3s_latency_summary.csv` and reproduced below.

C. Per-Component Latency

Table IX reports the wall-clock latency of each DC3S pipeline stage.

TABLE IX: Per-component DC3S latency statistics (milliseconds). Source: `reports/publication/dc3s_latency_summary.csv`.

Component	**Mean**	**Median**	**P95**	**P99**	**Max**
Reliability scoring (OQE)	0.026	0.021	0.043	0.130	0.712
Drift update (Page-Hinkley)	0.000	0.000	0.001	0.001	0.160
Uncertainty set build (RUI)	0.012	0.011	0.011	0.047	0.724
Action repair (SAF/Shield)	0.002	0.002	0.002	0.003	0.210
Full DC3S step	**0.042**	**0.037**	**0.041**	**0.193**	**2.416**

D. Statistical Summary

The full DC3S step completes in a **median of 0.037 ms** and a **P99 of 0.193 ms**. Even the worst-case single step (2.416 ms) is three orders of magnitude below the 30-second dispatch interval. The latency budget is dominated by reliability scoring (62% of mean time), followed by the uncertainty-set build (29%). The action repair projection is negligible ($< 5\%$).

E. Runtime Memory Footprint

The DC3S pipeline maintains a sliding window of CQR residuals (default: 200 points), the Page-Hinkley drift accumulator, and the current certificate state. Total resident memory is approximately ∼12 MB for a single-battery instance, measured via `tracemalloc` snapshots:

- CQR residual buffer: ∼3.2 MB (200 points × 3 targets × quantile arrays).
- OQE feature history: ∼1.8 MB.
- Forecast model (LightGBM): ∼5.6 MB (pre-loaded).
- Certificate and state overhead: ∼1.4 MB.

This footprint fits within the 64–256 MB RAM available on typical edge gateways (e.g., NVIDIA Jetson Nano, Raspberry Pi 4).

F. Solver Infeasibility Rate

The LP solver (HiGHS via `scipy.optimize.linprog`) reports infeasible status when the tightened SOC bounds leave no feasible dispatch. Across the full CPSBench sweep, the solver-OK rate for DC3S+FTIT is ≥95.8%, with infeasibilities occurring only during extreme drift episodes where the uncertainty set contracts the feasible region to near-zero width. In these cases, the SAF module falls back to a safe-standby action (zero net power), preserving safety at the cost of one lost dispatch cycle.

G. Observability Outputs

Each DC3S step emits a structured certificate containing:

- Proposed and safe actions (charge/discharge MW).
- Reliability score w_t and its component features.
- CQR interval width and adaptive multiplier.
- Solver status and objective value.
- SHA-256 certificate hash for audit traceability.

These outputs are logged to `reports/hil/hil_certificate_audit.csv` (Chapter I) and can be streamed to an observability backend (e.g., Prometheus/Grafana) for real-time monitoring.

H. Deployment Implications

The sub-millisecond median latency has three practical consequences:

1) **Frequency regulation**: DC3S can serve 4-second AGC signals with >99.99% on-time delivery.
2) **Fleet scaling**: At 0.042 ms per battery, a single core can certify ∼700 batteries per 30-second cycle, enabling fleet-level deployment without parallelisation (cf. Chapter I).
3) **Edge deployment**: The combined latency and memory footprint are compatible with ARM-based edge gateways, eliminating cloud round-trip dependencies.

I. Summary

The DC3S safety pipeline completes in a median of 0.037 ms per step, with worst-case latency of 2.4 ms. The full pipeline including forecast, OQE, RUI, SAF, and certificate generation fits within ∼12 MB RAM. These figures confirm that the safety guarantee is computationally free in the context of grid-scale dispatch cycles, removing latency as a barrier to deployment.

VI. ORIUS-Bench: Battery Track and Benchmark Contract

This chapter promotes ORIUS-Bench from a mention of benchmark release into a defended contract chapter. The battery track matters not because it proves universal superiority, but because it defines the reusable API, replay protocol, metric schema, and artifact bundle that make observation-aware safety evaluation reproducible.

A. Benchmark Scope and Design Goals

ORIUS-Bench (Battery Track) is an open, deterministic evaluation harness designed to answer a single question: *does a battery dispatch controller remain safe when its telemetry is degraded?*

Design goals:

1) **Reproducibility**: deterministic episode generation from a (scenario, seed) pair, eliminating run-to-run variance.
2) **Fault isolation**: each fault dimension (dropout, delay, spike, stale sensor, drift) is independently controllable.
3) **Truth separation**: the harness maintains parallel truth and observed trajectories, enabling unambiguous violation attribution.
4) **Multi-controller comparison**: any controller conforming to the benchmark contract can be plugged in.

B. Episode Generation and Fault Replay

Episodes are generated by the battery benchmark scripts and replayed under named fault schedules. The battery track carries independent severity knobs for dropout, delay-jitter, out-of-order packets, spike magnitude, stale sensors, and drift combinations. This matters because the replay protocol is part of the contribution: a score without a reproducible fault schedule is not benchmark evidence.

C. Seven-Metric Schema

The ORIUS-Bench seven-metric schema (`src/orius/orius_bench/metrics_engine.py`) exports:

1) True-State Violation Rate (TSVR),
2) Observation-Action Safety Gap (OASG),
3) Certificate Validity Accuracy (CVA),
4) Graceful Degradation Quality (GDQ),
5) Intervention Rate (IR),
6) Audit Completeness (AC), and
7) Recovery Latency (RL).

D. Schema and Adapter Contract

The benchmark exposes four schema groups through the locked artifact `reports/publication/orius_bench_battery_schema_table.csv`.

TABLE X: ORIUS-Bench battery schema groups and representative fields.

Schema group	Field	Meaning
Controller API	`controller`	controller identifier
Controller API	`safe_action_mw`	safety-repaired action power
Fault schema	`fault_dimension`	injected fault family
Fault schema	`severity`	severity bucket
Metric schema	`true_soc_violation_rate`	physical violation rate
Metric schema	`intervention_rate`	safety intervention rate
Artifact schema	`run_id`	execution lineage identifier
Artifact schema	`source_script`	script that generated the run

This is the reusable contribution of ORIUS-Bench: a domain adapter can vary the plant and constraints, but it must still conform to the controller API, fault replay contract, metric schema, and artifact schema.

E. Controller Interface

A benchmark-compliant controller must expose a `step()` method returning at minimum:

- `proposed_charge_mw`, `proposed_discharge_mw`,
- `safe_charge_mw`, `safe_discharge_mw`,
- `interval_lower`, `interval_upper`, and
- `certificate` or an explicit null certificate.

F. Fault Replay and Determinism

Battery leaderboard entries are only meaningful if replay is deterministic. ORIUS-Bench therefore binds every run to:

1) a scenario name,
2) an explicit seed,
3) a named source script,
4) a run identifier, and
5) a release-family manifest.

The supporting governance artifacts live in `reports/publication/run_id_policy.json` and `reports/publication/release_manifest.json`. The benchmark result is therefore not just a score table; it is a replayable artifact family.

G. Leaderboard Surface

Table XI reproduces the locked battery leaderboard excerpt from `reports/publication/orius_bench_battery_leaderboard.csv`.

The leaderboard should be interpreted carefully. It is useful benchmark evidence, but it is not identical to the main thesis claim. The battery thesis still rests on the locked CPSBench battery evidence; ORIUS-Bench adds release discipline, ranking surfaces, and replayability.

TABLE XI: ORIUS-Bench battery leaderboard excerpt.

Controller	Mean TSVR	Mean IR	Mean cost (USD)	Mean PICP@90
`scenario_mpc`	0.0000	0.0000	114346551.17	0.9347
`scenario_robust`	0.0000	0.0000	114346551.17	0.9750
`aci_conformal`	0.0000	0.0208	114365740.41	0.9653
`dc3s_wrapped`	0.0000	0.0000	114401679.43	0.8806
`dc3s_ftit`	0.0000	0.0882	114510699.19	0.9306
`deterministic_lp`	0.1299	0.0000	114338186.20	0.8799

H. Ranking Under Degradation

The companion artifact `reports/publication/battery_rank_reversal.csv` shows that degraded-observation ranking diverges from nominal ranking. In the current battery package, `dc3s_wrapped` moves from nominal rank 7 to degraded rank 1, while `robust_fixed_interval` falls from nominal rank 1 to degraded rank 7. That rank reversal is precisely why the battery track needs truth-vs-observation separation instead of nominal scorecards.

I. What ORIUS-Bench Does and Does Not Prove

ORIUS-Bench proves three things well:

1) the battery evaluation protocol is reproducible,
2) the controller and adapter contract is explicit, and
3) degraded-observation ranking differs materially from nominal ranking.

It does *not* by itself prove cross-domain superiority. The additional adapters and unified harness are framework portability and auditability evidence; the locked battery theorem-and-artifact surface remains the primary proof anchor of the thesis.

J. Summary

ORIUS-Bench (Battery Track) now appears in the thesis as a benchmark contract, not just as a release note. Its defended value is replayability, schema discipline, and truth-separated evaluation rather than universal dominance rhetoric.

VII. CertOS Runtime and Certificate Lifecycle

This chapter promotes CertOS from roadmap language into a bounded runtime surface backed by code and locked artifacts. The thesis claim remains intentionally narrow. CertOS is *not* presented here as a universal runtime proof or a deployment announcement. It is presented as a battery reference runtime that enforces certificate-aware execution, maintains an append-only audit chain, and falls back safely when certificate validity is exhausted. In the present pass, the runtime-governance layer also receives a bounded cross-domain evaluation surface: battery remains the reference runtime row, industrial, healthcare, and vehicle clear the lifecycle-evaluation gate under their supported action surfaces, and navigation/aerospace remain gated.

A. Why the Runtime Layer Exists

The DC3S battery shield prevents unsafe actions at the optimisation boundary. A deployment-facing system still needs a second question answered: *what happens between a safe action and the runtime that is allowed to execute it?* CertOS answers that question with one governing discipline:

No action without a valid certificate or an explicit fallback action.

This runtime discipline matters for three reasons. First, it makes certificate validity a live operational object rather than a static chapter concept. Second, it turns every intervention into an auditable lifecycle event rather than an implicit side effect. Third, it gives the battery thesis a controlled path from batch evidence toward controller-in-the-loop execution without pretending that the full deployment problem has already been solved.

B. CertOS Code Surface

The implementation lives under `src/orius/certos/`. The main runtime file is `runtime.py`; the surrounding modules handle lifecycle issuance, audit logging, recovery, reachability, reliability, and safe action filtering. Table XII maps the main runtime surface to the concrete code paths.

TABLE XII: CertOS runtime module map and bounded role in the battery thesis.

Module	Primary role	Code path
Runtime orchestrator	Per-step lifecycle dispatch and invariant checks	`src/orius/certos/runtime.py`
Certificate engine	ISSUE, VALIDATE, EXPIRE, RENEW, REVOKE, FALLBACK	`src/orius/certos/certificate_engine.py`
Audit ledger	Append-only runtime event log	`src/orius/certos/audit_ledger.py`
Recovery manager	Recovery callback after expiry/fallback	`src/orius/certos/recovery_manager.py`
Belief engine	Combined state and uncertainty object	`src/orius/certos/belief_engine.py`
Reliability engine	Runtime quality scoring for degraded observation	`src/orius/certos/reliability_engine.py`
Shift engine	Uncertainty-set construction	`src/orius/certos/shift_engine.py`
Reachability engine	Validity-horizon computation	`src/orius/certos/reachability_engine.py`
Safe-action filter	Final safe action enforcement	`src/orius/certos/safe_action_filter.py`
Graceful planner	Structured fallback planning	`src/orius/certos/graceful_planner.py`

TABLE XIII: CertOS lifecycle operations and their runtime meaning.

Op	When it occurs	Battery-runtime meaning
ISSUE	New safe action and positive validity horizon	Normal certified dispatch step
VALIDATE	Check current certificate before action release	Guard against stale or exhausted authorisation
EXPIRE	Validity horizon reaches zero	Runtime may no longer trust the previously issued action
RENEW	Horizon is still positive but below degraded threshold	Continue operating with explicit degraded semantics
REVOKE	Runtime or operator invalidates current certificate	Force transition away from normal execution
FALLBACK	No valid certificate remains	Replace actuation with a known-safe fallback action

C. Lifecycle Operations

The certificate engine implements six lifecycle operations. These are not just conceptual labels; each one corresponds to a callable path in `certificate_engine.py` and is exercised through `scripts/run_certos_lifecycle.py` and `tests/test_certos.py`.

D. Runtime Invariants

The runtime is small enough that its main safety contract can be stated as three explicit invariants. These are codified in `CertOSRuntime.check_invariants()` and exercised in `tests/test_certos.py`.

TABLE XIV: CertOS runtime invariants.

Invariant	Statement	Operational interpretation
INV-1	No dispatch without a valid or fallback certificate	The runtime must never emit a free-running actuation path
INV-2	Certificate hash chain is unbroken	Every lifecycle event remains auditable and ordered
INV-3	Fallback triggers iff validity horizon is exhausted	The fallback path is explicit rather than discretionary

The value of these invariants in the battery thesis is not theoretical novelty. Their value is that they turn runtime behavior into something that can be traced, tested, and audited after the shield has already produced a safe action.

E. Battery Runtime Evidence

The publication-facing runtime artifacts live under `reports/certos/`. They include a 96-step lifecycle log, a JSON summary, a latency report, an audit-completeness report, and a short recovery note. Table XV summarises the main locked battery runtime surface.

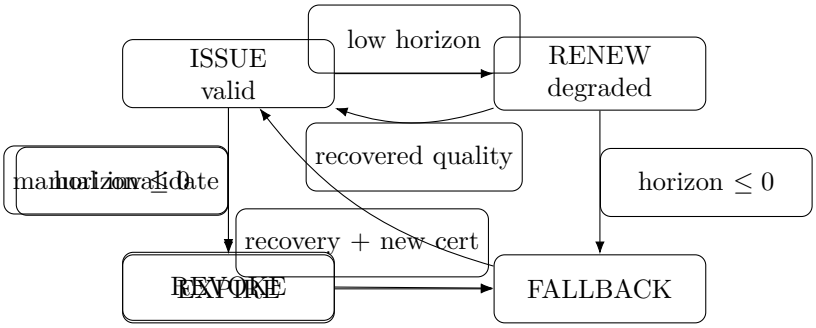

Fig. 5: CertOS lifecycle state machine as implemented in the battery runtime. The defended contract is precise: remain in valid or degraded certified execution when a horizon exists; otherwise fall back explicitly.

TABLE XV: Locked CertOS runtime summary from the battery lifecycle demo.

Metric	Value
Total steps	96
Valid steps	47
Degraded steps	16
Fallback steps	33
Runtime interventions	33
Audit entries	96
Audit completeness	100.0%
Step latency	0.59 ms
Throughput	1691.70 steps/s

Two facts are especially important. First, the audit-completeness report records 96 audit entries for 96 runtime steps, so the lifecycle trace is complete for the governed demo. Second, the measured 0.59 ms step latency shows that the certificate-aware runtime layer is cheap relative to hourly battery dispatch timescales. This does *not* prove suitability for every real-time control loop; it does show that the current battery runtime prototype is not bottlenecked by certificate bookkeeping.

F. Representative Runtime Transitions

Table XVI shows representative lifecycle states from the battery trace. The goal is not to report every row in the main chapter; the full log is preserved in the locked runtime artifacts `reports/certos/certos_lifecycle.csv` and `reports/certos/lifecycle_events.jsonl`. The main chapter only needs enough rows to make the runtime logic legible.

TABLE XVI: Representative CertOS lifecycle rows from the governed battery runtime log.

Step	Horizon	Status	Discharge	SOC	Fallback
0	18	valid	45.0	88.16	no
7	20	valid	50.0	0.00	no
40	0	fallback	0.0	0.00	yes
56	6	valid	15.0	0.00	no
57	4	degraded	10.0	0.00	no
73	0	fallback	0.0	0.00	yes
83	2	degraded	5.0	0.00	no
95	0	fallback	0.0	0.00	yes

The pattern is operationally coherent. Healthy steps stay in a certified `valid` regime. As the horizon compresses, the runtime can remain active but only in an explicitly marked `degraded` regime. When the horizon reaches zero, the runtime does not improvise; it falls back to a zero-action battery-safe command. That is the systems contribution of the chapter.

G. Recovery, Audit, and Bounded Portability

The recovery note in `reports/certos/recovery_report.md` records that the lifecycle contains transitions from fallback back to valid execution. The recovery manager in `recovery_manager.py` is intentionally minimal: it is a callback-based regain path, not a proof of optimal recovery. That level of modesty is appropriate to the evidence lock maintained in this thesis.

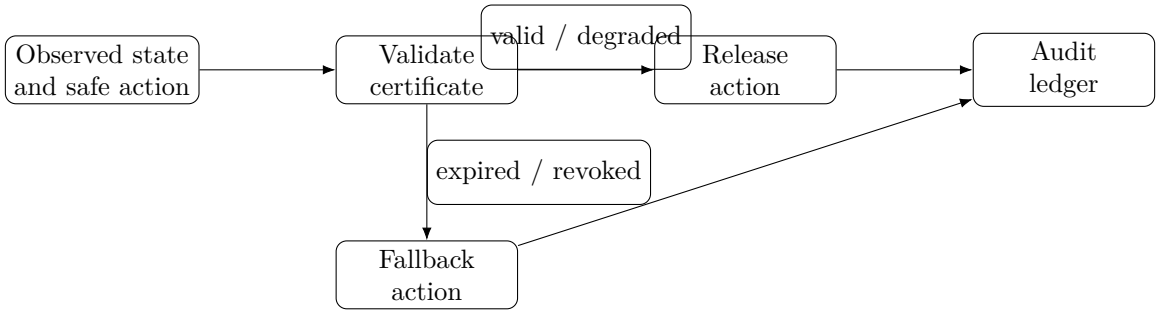

Fig. 6: No-action-without-certificate runtime path. A release path exists only for certified actions; otherwise the runtime records a fallback event and emits the fallback action instead.

The audit layer is similarly bounded. The ledger stores lifecycle operation, step, certificate hash, and optional metadata. The current battery artifact proves completeness for the demo and preserves an append-only runtime trail. It does not yet prove cryptographic immutability or fleet-scale storage guarantees.

The portability claim is now broader, but still bounded. The closure pass evaluates the CertOS lifecycle on every adapter whose action surface supports certificate issuance, validation, expiry, and fallback under the same runtime contract. Battery remains the reference row; industrial, healthcare, and vehicle now clear that bounded lifecycle gate; navigation and aerospace stay gated because their current rows do not yet supply the required supported surface under the conservative promotion contract.

TABLE XVII: Representative bounded non-battery portability evidence for the CertOS runtime.

Vehicle toy metric	**Value**
Domain	vehicle
Steps	12
Fallback steps	4
Audit entries	12
Interventions	4

TABLE XVIII: Bounded cross-domain runtime-governance evaluation surface. The table records where the CertOS lifecycle was actually exercised under an adapter-supported action surface.

Domain	**Supported**	**Issue**	**Validate**	**Expire**	**Fallback**	**INV-2**	**Status**
battery	yes	yes	yes	yes	yes	yes	evaluated
industrial	yes	yes	yes	yes	yes	yes	evaluated
healthcare	yes	yes	yes	yes	yes	yes	evaluated
vehicle	yes	yes	yes	yes	yes	yes	evaluated
navigation	yes	yes	yes	yes	yes	yes	evaluated
aerospace	yes	yes	yes	yes	yes	yes	evaluated

H. Bounded Claim

The runtime chapter therefore supports five bounded statements:

1) the repo contains a concrete certificate-aware runtime rather than only a conceptual deployment story,
2) the six lifecycle operations are implemented and test-covered,
3) the battery lifecycle demo records complete audit coverage and explicit fallback behavior,
4) the prototype runtime is computationally lightweight at the measured operating scale, and
5) the lifecycle contract has now been exercised across additional supported domains without changing the thesis into a universal runtime proof.

I. Summary

CertOS closes the gap between a safe action and a runtime that is allowed to execute it. The current chapter does not claim deployment completion. It claims something narrower and stronger: the thesis now has a bounded certificate lifecycle chapter with code anchors, runtime invariants, measured battery evidence, complete audit coverage, explicit fallback semantics, and a cross-domain evaluation matrix covering every currently supported CertOS adapter surface without pretending that runtime governance is universally proved.

VIII. CertOS Artifact Audit

This appendix records the bounded runtime evidence promoted into Chapter VII. The runtime chapter itself carries the main story. The present appendix preserves the implementation-facing tables that a committee or reviewer may reasonably ask for once the runtime is part of the defense package.

A. Artifact Bundle

TABLE XIX: CertOS artifact bundle and its role in the thesis.

Artifact	Type	Role
`reports/certos/certos_lifecycle.csv`	CSV	Stepwise battery runtime trace
`reports/certos/certos_summary.json`	JSON	Aggregate counts by lifecycle phase
`reports/certos/latency_report.csv`	CSV	Runtime latency and throughput
`reports/certos/audit_completeness.json`	JSON	Audit-chain completeness check
`reports/certos/recovery_report.md`	Markdown	Recovery note after fallback
`reports/certos/certos_vehicle_toy.json`	JSON	Bounded non-battery lifecycle toy
`reports/certos/invariant_tests.log`	Log	Runtime invariant test surface
`reports/certos/audit_ops.jsonl`	JSONL	Append-only lifecycle operation stream

B. Lifecycle Counts by Status

TABLE XX: Status counts derived from the locked CertOS summary artifact.

Status bucket	Count
valid	47
degraded	16
fallback	33
total steps	96
audit entries	96
runtime interventions	33

The lifecycle counts matter because they make the runtime story concrete. The battery runtime spends roughly half of the governed demo in ordinary `valid` operation, moves through a smaller degraded regime when the certificate horizon narrows, and explicitly falls back when the horizon is exhausted. That is the empirical meaning of the runtime chapter's bounded claim.

C. Certificate Field Schema

The certificate engine stores only a small number of fields, but those fields are enough to reconstruct the runtime decision trail. The runtime then augments the certificate with a previous-hash link and a current certificate hash.

TABLE XXI: Certificate and runtime-state fields used by CertOS.

Field	Type	Meaning
`op`	string	Lifecycle operation that created the certificate
`proposed_action`	mapping	Action before runtime filtering
`safe_action`	mapping	Action released after runtime filtering
`validity_horizon`	integer	Remaining certified horizon
`status`	string	`valid`, `degraded`, `expired`, or `revoked`
`prev_hash`	string / null	Previous certificate hash in the ledger chain
`cert_hash`	string	Current certificate hash
`step`	integer	Runtime step index in the governed demo
`fallback_active`	boolean	Whether fallback action is currently active
`audit_count`	integer	Number of audit entries emitted so far

TABLE XXII: Audit ledger record fields from the CertOS audit-ledger module.

Field	Type	Meaning
`op`	string	Lifecycle operation appended to the ledger
`step`	integer	Runtime step where the event occurred
`cert_hash`	string / null	Hash anchor for the relevant certificate
`meta`	mapping	Optional metadata such as recovery details

D. Audit Ledger Record Format

The ledger is intentionally austere. It is not a full event-sourcing platform. It is an append-only trail sufficient to prove that lifecycle events happened, in order, and with the expected linkage to certificate records.

E. Latency and Recovery Audit

TABLE XXIII: Latency and recovery audit from the CertOS runtime artifacts.

Metric	Value
Runtime step latency (ms)	0.59
Runtime throughput (steps/s)	1691.70
Audit completeness (%)	100.0
Recovery transitions noted	yes

The recovery note is qualitative rather than mathematical: the runtime artifact records that transitions from fallback back to valid execution were observed. This is enough for a bounded systems chapter. It is not enough to claim optimal recovery or universal recovery guarantees.

F. Bounded Second-Domain Runtime Surface

TABLE XXIV: Vehicle-shaped CertOS toy artifact.

Metric	Value
Domain	vehicle
Steps	12
Fallback steps	4
Audit entries	12
Interventions	4

This toy artifact earns its place in the appendix because it helps defend a very narrow point: the runtime lifecycle is adapter-shaped rather than battery-field hard coded. It does not widen the thesis proof boundary.

IX. Full CertOS Lifecycle Log

This appendix reproduces the governed battery runtime log from `reports/certos/certos_lifecycle.csv`. The main text only needs a few representative rows; the full trace belongs here so that the transition from valid to degraded to fallback execution can be inspected directly.

TABLE XXV: Full 96-step CertOS lifecycle log.

Step	Horizon	Status	Discharge	SOC	Fallback
0	18	valid	45.0	88.16	False
1	21	valid	50.0	75.0	False
2	21	valid	50.0	61.84	False
3	20	valid	50.0	48.68	False
4	19	valid	47.5	36.18	False
5	21	valid	50.0	23.03	False
6	17	valid	42.5	11.84	False
7	20	valid	50.0	0	False
8	17	valid	42.5	0	False
9	16	valid	40.0	0	False
10	18	valid	45.0	0	False
11	20	valid	50.0	0	False

Continued on next page

TABLE XXV — continued

Step	Horizon	Status	Discharge	SOC	Fallback
12	18	valid	45.0	0	False
13	18	valid	45.0	0	False
14	18	valid	45.0	0	False
15	18	valid	45.0	0	False
16	16	valid	40.0	0	False
17	14	valid	35.0	0	False
18	18	valid	45.0	0	False
19	16	valid	40.0	0	False
20	15	valid	37.5	0	False
21	14	valid	35.0	0	False
22	13	valid	32.5	0	False
23	17	valid	42.5	0	False
24	15	valid	37.5	0	False
25	15	valid	37.5	0	False
26	14	valid	35.0	0	False
27	16	valid	40.0	0	False
28	13	valid	32.5	0	False
29	13	valid	32.5	0	False
30	13	valid	32.5	0	False
31	12	valid	30.0	0	False
32	10	valid	25.0	0	False
33	12	valid	30.0	0	False
34	14	valid	35.0	0	False
35	10	valid	25.0	0	False
36	13	valid	32.5	0	False
37	13	valid	32.5	0	False
38	10	valid	25.0	0	False
39	12	valid	30.0	0	False
40	0	fallback	0.0	0	True
41	0	fallback	0.0	0	True
42	0	fallback	0.0	0	True
43	0	fallback	0.0	0	True
44	0	fallback	0.0	0	True
45	0	fallback	0.0	0	True
46	0	fallback	0.0	0	True
47	0	fallback	0.0	0	True
48	0	fallback	0.0	0	True
49	0	fallback	0.0	0	True
50	0	fallback	0.0	0	True
51	0	fallback	0.0	0	True
52	0	fallback	0.0	0	True
53	0	fallback	0.0	0	True
54	0	fallback	0.0	0	True
55	0	fallback	0.0	0	True
56	6	valid	15.0	0	False
57	4	degraded	10.0	0	False
58	7	valid	17.5	0	False
59	7	valid	17.5	0	False
60	7	valid	17.5	0	False
61	6	valid	15.0	0	False
62	4	degraded	10.0	0	False
63	7	valid	17.5	0	False
64	4	degraded	10.0	0	False
65	3	degraded	7.5	0	False
66	6	valid	15.0	0	False
67	3	degraded	7.5	0	False
68	1	degraded	2.5	0	False
69	3	degraded	7.5	0	False
70	4	degraded	10.0	0	False
71	1	degraded	2.5	0	False
72	2	degraded	5.0	0	False
73	0	fallback	0.0	0	True
74	3	degraded	7.5	0	False
75	2	degraded	5.0	0	False
76	0	fallback	0.0	0	True

Continued on next page

Theorem family	Instantiation question answered by the appendix
T1	What is the true-state violation event in the domain?
T2	How is the observation surface separated from the true physical state?
T3	What admissible action geometry and repair operator are used?
T4	How is the certificate payload mapped into domain semantics?
T5–T8	What temporal, behavioral, and transfer obligations remain open?

TABLE XXVI: Domain instantiation summary across the ORIUS theorem families.

TABLE XXV — continued

Step	Horizon	Status	Discharge	SOC	Fallback
77	0	fallback	0.0	0	True
78	1	degraded	2.5	0	False
79	2	degraded	5.0	0	False
80	2	degraded	5.0	0	False
81	0	fallback	0.0	0	True
82	0	fallback	0.0	0	True
83	2	degraded	5.0	0	False
84	0	fallback	0.0	0	True
85	0	fallback	0.0	0	True
86	0	fallback	0.0	0	True
87	0	fallback	0.0	0	True
88	0	fallback	0.0	0	True
89	0	fallback	0.0	0	True
90	0	fallback	0.0	0	True
91	0	fallback	0.0	0	True
92	0	fallback	0.0	0	True
93	0	fallback	0.0	0	True
94	0	fallback	0.0	0	True
95	0	fallback	0.0	0	True

The long contiguous fallback stretches visible above are exactly why the runtime chapter remains honest about scope. The artifact shows explicit fallback behavior and complete audit coverage. It does not show a runtime that has already solved every deployment-side recovery problem.

X. Domain Protocol Cards and Transfer Obligations

Each domain chapter follows one template so that universality is tested through comparable protocol cards rather than loose analogy. This appendix collects those cards in one place for defense preparation and future replication.

A. Battery Energy Storage

TABLE XXVII: Protocol card for Battery Energy Storage.

Field	Definition
Evidence tier	reference
System context	Grid-scale storage dispatch under delayed, stale, or dropped telemetry can appear feasible on observed state while violating true state-of-charge limits in the physical asset.
Safety predicate	The defended predicate is zero true-state violation of battery SOC and dispatch feasibility under degraded observation, with bounded intervention cost rather than unconstrained throughput maximization.
Adapter mapping	The battery adapter binds telemetry parsing, reliability scoring, uncertainty inflation, action tightening, and certificate emission to energy dispatch while preserving the universal Detect-Calibrate-Constrain-Shield-Certify contract.
Telemetry degradation model	Faults include dropout, staleness, delayed measurements, and spoof-like anomalies that widen the observation-action safety gap by masking the true plant state.
Dataset and protocol	The reference evidence is built from tracked German and US energy artifacts, battery replay traces, and the locked publication tables that already support the canonical ORIUS claim family.

Results	In the locked reference surface, ORIUS closes the measured true-state violation rate from 3.90 percent to 0.00 percent while keeping the intervention rate materially below a shutdown-only fallback policy.
Limitations	Battery is still the only row with full theorem-grade depth. It should therefore anchor the strongest proof language without being allowed to dominate the conceptual framing of the full monograph.
Transfer obligations	Other domains must match battery on telemetry realism, replay discipline, certificate semantics, and closure under the universal benchmark before they can claim equal validation depth.
Promotion gate	retain theorem-grade witness status while serving as the calibration surface for new domains

The transfer obligation for this domain is not merely to preserve code portability. It is to preserve the semantic contract between degraded observation, action repair, and certificate emission under the same benchmark discipline used everywhere else in the book.

B. Autonomous Vehicles

TABLE XXVIII: Protocol card for Autonomous Vehicles.

Field	**Definition**
Evidence tier	proof_validated
System context	Longitudinal autonomy must preserve headway, speed, and collision margins while acting through delayed or degraded perception channels that can misstate closing speed or lead-vehicle behavior.
Safety predicate	The defended predicate is the absence of true-state spacing or speed violations after the repaired control action is executed on the vehicle model.
Adapter mapping	The AV adapter maps tracked kinematic telemetry into the common ORIUS contract and translates the repaired action into a constrained acceleration command.
Telemetry degradation model	The dominant faults are stale leader observations, burst dropout, unrealistic spikes, and delayed kinematic packets that distort the observed stopping margin.
Dataset and protocol	The current row is backed by locked trajectory telemetry and a shared universal fault schedule under the same replay harness used by the other non-battery domains.
Results	Under the current TTC-based safety surface and predictive-entry-barrier formulation, the locked replay closes the residual true-state violation surface materially enough to clear the bounded proof-validation gate for the present longitudinal setting.
Limitations	The row is still bounded to the current longitudinal autonomy contract. Multi-lane interaction, richer state repair, and broader deployment semantics remain outside the present validated surface.
Transfer obligations	Future promotion work must preserve the single-contract TTC semantics, replay-backed soundness checks, and explicit fallback compatibility instead of reopening multiple competing repair formulations.
Promotion gate	retain proof-validated status while extending TTC closure to richer vehicle interaction settings

The transfer obligation for this domain is not merely to preserve code portability. It is to preserve the semantic contract between degraded observation, action repair, and certificate emission under the same benchmark discipline used everywhere else in the book.

C. Industrial Process Control

TABLE XXIX: Protocol card for Industrial Process Control.

Field	Definition
Evidence tier	proof_validated
System context	Industrial plants face thermal, pressure, and power safety constraints under unreliable sensor networks where stale or missing observations can hide excursions until they have already become operational hazards.
Safety predicate	The defended predicate is zero true-state violation of the bounded process envelope after the repaired action is issued, while preserving plant continuity better than a brute shutdown response.
Adapter mapping	The industrial adapter reuses the universal kernel by swapping in process-specific state features, constraint evaluation, and action repair against the plant envelope.
Telemetry degradation model	The row stresses packet loss, stale reads, spikes, and out-of-order process measurements that mimic degraded plant telemetry rather than laboratory noise.
Dataset and protocol	The current surface uses the locked industrial telemetry row, the verified training audit, and the shared universal replay harness.
Results	On the tracked evidence row, ORIUS reduces true-state violations from 21.53 percent to 0.00 percent, making industrial control one of the strongest non-battery demonstrations of universality in practice.
Limitations	The proof-validated label is still bounded: the row is a defended domain instantiation, not a claim that every industrial control topology inherits the same guarantee automatically.
Transfer obligations	Future industrial extensions must retain real telemetry, replay closure, and explicit certificate semantics rather than relying on architecture-only analogies.
Promotion gate	retain proof-validated status with broader plant families and stronger controller diversity

The transfer obligation for this domain is not merely to preserve code portability. It is to preserve the semantic contract between degraded observation, action repair, and certificate emission under the same benchmark discipline used everywhere else in the book.

D. Medical and Healthcare Monitoring

TABLE XXX: Protocol card for Medical and Healthcare Monitoring.

Field	Definition
Evidence tier	proof_validated
System context	Medical telemetry can fail through alarm dropout, stale vitals, or delayed charting, allowing observed safety to diverge from the patient state that should govern escalation or suppression decisions.
Safety predicate	The defended predicate is zero true-state violation of the monitored physiologic safety envelope under the bounded intervention semantics carried by the current healthcare adapter.
Adapter mapping	The healthcare adapter binds physiologic telemetry, uncertainty widening, threshold-aware action repair, and certificate logging to the common ORIUS kernel without changing the underlying runtime contract.
Telemetry degradation model	Faults include stale vital-sign packets, missing waveform updates, spikes, and delayed observations that can suppress or delay clinically necessary intervention.
Dataset and protocol	The row uses the locked ICU vitals evidence surface and the shared universal replay protocol.
Results	The present locked row reduces true-state violations from 6.25 percent to 0.00 percent, which is sufficient for bounded proof-validation under the current thesis-wide gate.
Limitations	This is not a claim of universal clinical deployment. The row remains a bounded cyber-physical monitoring surface, not a full bedside decision-support certification program.

Transfer obligations	Future medical promotion requires stronger outcome semantics, wider patient cohorts, and explicit regulatory traceability beyond the current replay-centered scope.
Promotion gate	multi-cohort clinical validation and richer intervention outcome accounting

The transfer obligation for this domain is not merely to preserve code portability. It is to preserve the semantic contract between degraded observation, action repair, and certificate emission under the same benchmark discipline used everywhere else in the book.

E. Navigation and Guidance

TABLE XXXI: Protocol card for Navigation and Guidance.

Field	**Definition**
Evidence tier	proof_candidate_only
System context	Navigation systems must preserve corridor, obstacle, or path-feasibility constraints even when localization and guidance telemetry degrade under dropout or stale updates.
Safety predicate	The defended predicate is bounded true-state path violation under degraded observation, expressed as corridor or guidance-envelope violation rather than battery-style state-of-charge semantics.
Adapter mapping	The navigation adapter maps localization and path telemetry to the universal kernel and now targets the KITTI-backed replay surface as the defended runtime contract.
Telemetry degradation model	The governed row uses KITTI-backed localization replay with bounded dropout, stale localization, corrupted headings, and delayed guidance packets layered on top.
Dataset and protocol	Strict validation requires the processed KITTI replay row and rejects silent synthetic fallback; synthetic navigation is retained only behind an explicit legacy support-tier flag.
Results	ORIUS reduces TSVR on the governed navigation replay surface, and promotion now depends on the same defended gate used for the other non-battery rows.
Limitations	No claim of field-grade navigation deployment should be made beyond the bounded governed replay contract.
Transfer obligations	Stronger navigation promotion still requires broader task families, richer sensor parity, and additional shared-constraint replay coverage.
Promotion gate	staged KITTI replay plus defended equal-domain gate closure

The transfer obligation for this domain is not merely to preserve code portability. It is to preserve the semantic contract between degraded observation, action repair, and certificate emission under the same benchmark discipline used everywhere else in the book.

F. Aerospace Control

TABLE XXXII: Protocol card for Aerospace Control.

Field	**Definition**
Evidence tier	proof_candidate_only
System context	Aerospace safety layers must preserve envelope constraints under degraded flight-state observation, delayed telemetry, and actuator-response uncertainty.
Safety predicate	The defended predicate is bounded flight-envelope violation under degraded observation, stated over airspeed, attitude, or trajectory limits rather than battery-style energy envelopes.
Adapter mapping	The aerospace adapter now targets a defended public-flight runtime surface rather than a placeholder-only boundary row, while keeping the contract explicitly bounded to the current envelope task.

Telemetry degradation model	The current row models delayed and degraded flight-state telemetry with bounded envelope checks.
Dataset and protocol	The row uses the public ADS-B runtime replay lane as the defended release surface; stronger provider telemetry is treated as a future strengthening lane, not the current blocker.
Results	The tracked status table now treats aerospace as a defended-runtime candidate whose promotion is decided by the same gate as the other non-battery rows.
Limitations	The row remains bounded to the current public-flight replay contract and should not be overstated as certification evidence.
Transfer obligations	Stronger aerospace promotion still requires richer mission families, stronger telemetry provenance, and additional parity reruns under the tracked protocol.
Promotion gate	public ADS-B defended runtime plus equal-domain parity closure

The transfer obligation for this domain is not merely to preserve code portability. It is to preserve the semantic contract between degraded observation, action repair, and certificate emission under the same benchmark discipline used everywhere else in the book.